

This homework has 9 questions, for a total of 100 points and 3 extra-credit points.

Unless otherwise stated, each question requires *clear*, *logically correct*, and *sufficient* justification to convince the reader.

We strongly encourage you to typeset your solutions in L^AT_EX.

If you collaborated with someone, you must state their name(s). You must *write your own solution* for all problems and *may not use any other student's write-up or any AI tools*.

0. **Before you start; before you submit.**

If applicable, state the name(s) and username(s) of your collaborator(s).

Solution:

- (10 pts) 1. **Self assessment.** Carefully read and understand the posted solutions to the previous homework. Identify one part for which your own solution has the most room for improvement (e.g., has unsound reasoning, doesn't show what was required, could be significantly clearer or better organized, etc.). Copy or screenshot this solution, then in a few sentences, explain what was deficient and how it could be fixed.

(Alternatively, if you think one of your solutions is significantly *better* than the posted one, copy it here and explain why you think it is better.)

If you didn't do last week's homework, choose a problem from it that looks challenging to you, and in a few sentences, explain the key ideas behind its solution in your own words.

Solution:

Grading notes:

2. **Counting strings!** Let $C(n)$ denote the number of binary strings of length n that have no consecutive occurrences of 1. For example, $C(3) = 5$; we see this by listing all binary strings of length 3 and checking that only the strings 000, 001, 010, 100, and 101 have no consecutive 1s.

- (2 pts) (a) Compute $C(5)$.

Solution: $C(5) = 13$.

+2 Correct computation for $C(5)$

- (4 pts) (b) Derive, with justification, a recurrence relation for $C(n)$. Remember to include appropriate base case(s).

Hint: Consider two cases depending on whether the first character is a 0 or a 1.

Solution: One base case is $C(0) = 1$: There is exactly one binary string of length 0 that have no consecutive occurrences of 1, the empty string.

The second base case is $C(1) = 2$: There are two binary strings of length 1 that have no consecutive occurrences of 1, the binary string 0 and 1.

As for the recursive case, observe that a string of length $n \geq 2$ that has no consecutive 1s must begin with either a 1 or a 0. If it begins with a 1, then next bit must be a 0, and the rest of the string can be any string of length $n - 2$ that has no consecutive 1s. So, there are $C(n - 2)$ such strings. If it begins with a 0, then there is no restriction on the next bit, so the rest of the string can be any string of length $n - 1$ that has no consecutive 1s. So, there are $C(n - 1)$ such strings.

The two cases are non-overlapping, so we conclude that $C(n) = C(n - 1) + C(n - 2)$ for $n \geq 2$.

Alternate Solution:

One base case is $C(0) = 1$: There is exactly one binary string of length 0 that have no consecutive occurrences of 1, the empty string.

The second base case is $C(1) = 2$: There are two binary strings of length 1 that have no consecutive occurrences of 1, the binary string 0 and 1.

The third base case is $C(2) = 3$: There are three binary strings of length 2 that have no consecutive occurrences of 1, the binary strings 01, 10, and 00.

For the recursive case, consider a binary string of length $n \geq 3$ with no consecutive 1s. Such a string must begin with either 0 or 1. In either case, the remaining $n - 1$ bits can form any valid string of length $n - 1$, giving $2C(n - 1)$ possibilities in total.

However, this count includes strings that begin with the invalid prefix 11. To see why, note that when the first bit is 1, we allowed the remaining $n - 1$ bits to be any valid string of length $n - 1$. But if that string itself begins with 1, then the overall string begins with 11, which violates the condition.

Any such string must actually begin with 110, followed by a valid string of length $n - 3$. There are $C(n - 3)$ of these, so we subtract them.

Thus the recurrence is $C(n) = 2C(n - 1) - C(n - 3)$.

+1 Correct computations for the base cases

+2 Correct recurrence relation

+1 Correct justification for the recurrence relation

- (3 pts) (c) Translate your recurrence into pseudocode that is *top-down recursive, without memoization*. You may use [Algorithm 26](#) in the course notes as an example. (Note that this algorithm runs in time exponential in the input value n .)

Finally, show a *lower bound* on the runtime of the algorithm, which establishes that the runtime is exponential in n .

(Note that since the input is n , the input *size* is $\Theta(\log n)$, so this exponential-in- n algorithm runs in *double exponential* time with respect to the input size, i.e. $a^{b^{\log n}}$ for some constants a and b .)

Solution:

Input: Non-negative integer n

```

1: function COUNT( $n$ )
2:   if  $n = 0$  then
3:     return 1
4:   if  $n = 1$  then
5:     return 2
6:   return COUNT( $n - 1$ ) + COUNT( $n - 2$ )

```

Let $T(n)$ denote the runtime of our function on input n . We can see that $T(n) = \Omega(2^{n/2})$, because

$$\begin{aligned}
 T(n) &\geq T(n-1) + T(n-2) && (T(n-1) \geq T(n-2)) \\
 &\geq T(n-2) + T(n-2) \\
 &= 2 \cdot T(n-2) \\
 &\geq 2 \cdot 2 \cdot T(n-4) \\
 &\vdots \\
 &\geq 2^k \cdot T(n-2k) && (\text{Via repeated substitution}) \\
 &= 2^{n/2} \cdot T(0) && (\text{At } k = n/2) \\
 &\geq \Omega(2^{n/2}) \cdot \Theta(1) = \Omega(\sqrt{2}^n).
 \end{aligned}$$

+1 Added correct base cases to the pseudocode

+1 Returns the recurrence relation in non-base cases. Recurrence should be “faithful” to the student’s answer to part (b)

+1 Correctly shows the lower bound on the runtime of the algorithm.

- (3 pts) (d) Translate your recurrence into dynamic-programming pseudocode. ([Algorithm 28](#) in the course notes is a good model.)

In this and subsequent DP questions, credit for pseudocode will be determined based only on how “faithful” (i.e., properly derived) it is to your recurrence—regardless of whether the recurrence itself is correct. (So, pseudocode that isn’t faithful to your recurrence will not get full credit—but pseudocode that properly matches an incorrect recurrence *will* get full credit.)

Additionally, for all DP questions, your pseudocode should be written in the “bottom-up” form, rather than top-down memoization.

Analyze the running time of your code, as a function of n . Note that unlike most problems, you are *not* being asked to write the running time in terms of the input size (n is the input value, not the input size.) You may use the fact (without proof) that $C(n)$ has $\Theta(n)$ bits,

so adding together two entries of the dynamic programming table takes $O(n)$ time.

Solution: The recurrence is the same as that for the Fibonacci sequence (but with different base cases). Since there are $O(n)$ cells in the table and each entry takes $O(n)$ time to compute, the total time complexity is $O(n^2)$.

Input: Non-negative integer n

```
1: function BOTTOM-UP-1( $n$ )
2:   Initialize an array  $A[0 \dots n]$ 
3:    $A[0] \leftarrow 1$ 
4:   if  $n = 0$  then
5:     return  $A[0]$ 
6:    $A[1] \leftarrow 2$ 
7:   for  $i = 2, \dots, n$  do
8:      $A[i] \leftarrow A[i - 1] + A[i - 2]$ 
9:   return  $A[n]$ 
```

Input: Non-negative integer n

```
1: function BOTTOM-UP-2( $n$ )
2:   Initialize an array  $A[0, 1]$ 
3:    $A[0] \leftarrow 1$ 
4:    $A[1] \leftarrow 2$ 
5:   for  $i = 2, \dots, n$  do
6:     Let  $A[i]$  be the least recently used array cell
7:      $A[i] \leftarrow A[0] + A[1]$ 
8:      $X \leftarrow A[0] + A[1]$  // Variable to hold last computed number
9:   return  $X$ 
```

+1 Base cases present in pseudocode

+1 Allocates and correctly fills out the table from index 0 to n

+1 Correct runtime analysis.

3. Seems Familiar?

In this course, pattern matching is a crucial skill. Many problems you will see on homeworks and exams are, at their cores, similar to those you have previously solved in lecture or discussion. Noticing these similarities allows you to use familiar strategies and techniques to solve new problems effectively.

In this problem, you are given 3 dynamic programming problems below. For each of the following parts, you are given another problem that is the same as one of aforementioned problems and can be solved with same recurrence.

For each part, state and briefly justify which problem it is a reformulation of. Explain which aspects of the two problems correspond to each other.

You do not need to provide the recurrence (though you may want to in order to justify your solution).

- **Knapsack Variant:** Given n items where each item i has an associated value v_i and weight w_i , and no two weights are the same, find a subset of items with maximum possible value, whose total weight is at most C .
- **Coin Change:** Given a list of coins $c = [c_1, \dots, c_k]$, where c_i is the value of coin i , and a target amount n , find the number of different collections of coins that have a total value of n .
- **Stair Climbing:** Given a staircase with n stairs, determine the number of distinct ways a person starting at the bottom can reach the top if each time they take a step they can advance either 1 or 2 stairs.

(4 pts)

- (a) **Pipe Cutting:** Given a pipe of length L , and a list of pipe lengths with associated prices, find the maximum revenue obtainable by cutting the pipe into pieces, and selling the pieces (discarding any leftover pipe), under the constraint that you may only sell each length of pipe a maximum of one time.

Solution: This is the same as the Knapsack Variant problem.

The optimization is the same. In both problems, we are maximizing the total value of the things we select. In the knapsack problem, we are selecting items, and in the pipe problem, we select which lengths of pipe to sell.

The constraints are the same as well. The parallels are as follows: The length L of the pipe is the same as the capacity C of our knapsack. The weights of the items in the knapsack problem correspond exactly to the lengths of the pipes. The values of the items in the knapsack problem are the same as the prices of each pipe length. Note that this equivalence only exists since we may only sell each length of pipe at most once.

+1 Correctly identify the problem

+1 Correct explanation about quantity being optimized

+2 Correct explanation about constraints

(4 pts)

- (b) **Domino Tiling:** Given a $2 \times n$ board, determine the number of ways to completely tile the board using 2×1 dominoes, which can be placed either vertically or horizontally. Every square of the board must be covered by precisely one domino.

Solution:

This is the same as the Stair Climbing problem. Note that both follow the Fibonacci sequence.

We are counting the same quantity: the number of ways to reach length n .

In Domino Tiling, we can either place a vertical domino, which leaves $n - 1$ remaining horizontal space, or place two horizontal dominoes, which leaves $n - 2$ remaining horizontal space. In stair climbing, we can either climb one step, leaving $n - 1$ steps to climb, or climb two steps, leaving $n - 2$ steps. Hence, both problems satisfy the recurrence $f(n) = f(n - 1) + f(n - 2)$.

+1 Correctly identify the problem

+3 Correct explanation about quantity being counted

4. Minimum Subarray Product

Given an array $A[1, \dots, n]$ of $n \geq 1$ *positive real numbers*, we are interested in finding the smallest product of any *nonempty contiguous* subarray of A , i.e.,

$$\min\{A[i]A[i + 1] \cdots A[j] : 1 \leq i \leq j \leq n\}.$$

- (8 pts) (a) In this problem you may assume for simplicity that multiplying a pair of real numbers can be done in $O(1)$ time. Give a recurrence relation, including base case(s), that would yield an $O(n)$ -time dynamic-programming algorithm for the smallest product problem. Briefly justify why your recurrence relation is correct. You will analyze the running time of the resulting algorithm in part (b).

Hint: The longest increasing subsequence recurrence might give you some inspiration (but this recurrence should be simpler than that one).

Solution: Recall that a *minimizing subarray* is a subarray (which may not be unique) that obtains the smallest possible product.

We can first make the simple observation that every minimizing subarray must end at some index in A . So, if we find the product of a minimizing subarray ending at index i for *every* $1 \leq i \leq n$, we can find an overall solution (the product of a minimizing subarray for A) by taking a best one of all these options, using $O(n)$ additional time.

We now describe a way to find the product of a minimizing subarray ending at index i . Recall that a subarray of an array is contiguous in A . This means that a minimizing subarray ending at index i must be either (1) a continuation of a minimizing subarray ending at index $i - 1$, or (2) the subarray consisting of the element $A[i]$ by itself. In the first case, the product of the subarray is the product of $A[i]$ and the product of the elements in a *minimizing* subarray ending at index $i - 1$. In the second case, the product is simply $A[i]$. The product of a minimizing subarray ending at index i is merely the minimum of these two possibilities.

Based on these observations, we define $S_{\text{at}}(i)$ to be the product of a minimizing subarray of A that ends at index i . For convenience, we define the product of an empty array to be 1, so the base case is $S_{\text{at}}(0) = 1$. However, it is also valid for the base case to be $A[1]$ for $i = 1$.

The recurrence for general i is: $S_{\text{at}}(i) = \min\{S_{\text{at}}(i - 1) \cdot A[i], A[i]\}$

It is also valid to break it into two cases to get $A[i]$ if $S_{\text{at}}(i - 1) > 1$, and $S_{\text{at}}(i - 1) \cdot A[i]$ otherwise.

Given this recurrence, we can compute the values of $S_{\text{at}}(i)$ for all i in a bottom-up fashion in $O(n)$ time, by allocating a table of size n , and for each $1 \leq i \leq n$, computing (and storing at table index i) the minimum of $S_{\text{at}}(i-1) \cdot A[i]$ and $A[i]$. (See the next part for the details.)

- +2 Correct base case
- +3 Correct recursive case
- +3 Correct justification of the recursive case

- (6 pts) (b) Give pseudocode implementing an $O(n)$ -time dynamic programming solution to the smallest product problem, and justify the running time. Recall that for simplicity, you may assume that multiplying a pair of real numbers can be done in $O(1)$ time. For all DP problems, remember to refer to the “DP Recipe” from lecture and discussion.

Solution:

```
function MINPROD( $A[1, \dots, n]$ )
    initialize array  $S[0, \dots, n]$ 
     $S[0] \leftarrow 1$ 
    for  $i = 1, \dots, n$  do
         $S[i] \leftarrow \min\{S[i-1] \cdot A[i], A[i]\}$ 
    return  $\min\{S[i] : 1 \leq i \leq n\}$ 
```

Runtime Analysis: Each element in the array takes constant amount of time to compute with respect to n as the algorithm performs only a fixed number of basic operations: it looks up one previously computed value in the array, multiplies it by $A[i]$, and compares the result with $A[i]$. Since this is $O(1)$ time per iteration and there are n iterations, the total running time of the algorithm is $O(n)$

- +1 Base case present in pseudocode
- +3 Pseudocode correctly assigns $S(i)$ based on the recurrence relation the student gives in part (a)
- +1 Solution runs in $O(n)$
- +1 Correct runtime analysis

5. Sugar Rush!

You and a friend have won first prize at a Halloween costume party (due to your brilliant Alan Turing and Alonzo Church costumes): a bucket of several pieces of candy, each having some amount of sugar. You and your friend plan to celebrate your win by staying up late and eating all of the candy, but first you need to divide it between yourselves. The problem is that if you

and your friend consume different amounts of sugar, the one who eats less sugar will crash first, leaving the other person to stay up alone. You need a way to divide the candy between the two of you so that you both eat the same amount of sugar, and crash at the same time.

In this problem you will give a dynamic-programming algorithm that, given an array $S = [s_1, s_2, \dots, s_n]$ where the i th piece of candy has a non-negative integer s_i grams of sugar, either divides the pieces into two ‘piles’ that have equal amounts of sugar, or (correctly) says that no such piles exist. (Each piece must be in exactly one of the piles.)

- (10 pts) (a) For a given array S , define $P(c, i)$ to be **true** if there exists a pile drawn from the first i pieces of candy having a total of c grams of sugar, and **false** otherwise. Derive, with justification, a recurrence relation for $P(c, i)$. Remember to include appropriate base case(s).

When writing recurrences with Boolean values, you may use operators such as $\wedge$ (and), $\vee$ (or), and $\neg$ (not).

Solution:

This problem is commonly known as the Partition Problem.

$$P(c, i) = \begin{cases} \text{True} & \text{if } c = 0, \\ \text{False} & \text{if } c > 0, i = 0 \\ P(c, i - 1) & \text{if } s_i > c \\ P(c - s_i, i - 1) \vee P(c, i - 1) & \text{otherwise.} \end{cases}$$

Justification:

- For the first base case, there is always a selection of numbers that sums to 0, namely, the empty selection, hence True.
- For the second base case, you cannot create a pile of positive sugar value $c > 0$ if you don’t have any candies $i = 0$, hence False.
- For the recursive cases: If $s_i > c$, then no selection from $S[1, \dots, i]$ containing s_i can sum to c , as all values are positive. So, the only possible selection from $S[1, \dots, i]$ that sums to c must have used values only from $S[1, \dots, i - 1]$.
- If $s_i < c$, a selection summing to c can be made by adding s_i to a selection from $S[1, \dots, i - 1]$ that has a sum of $c - s_i$ or by using a selection from $S[1, \dots, i - 1]$ that already sums to c .

Alternate solution: We can use the “out of bounds” condition as a separate base case. The base case and recurrence for P is

$$P(c, i) = \begin{cases} \text{False} & \text{if } c < 0, \\ \text{True} & \text{if } c = 0, \\ \text{False} & \text{if } c > 0, i = 0 \\ P(c - s_i, i - 1) \vee P(c, i - 1) & \text{otherwise.} \end{cases}$$

Justification for the new base case: there is no selection of numbers $S[i]$ with a negative sum, because they are all non-negative, hence False.

Justification for other base cases is present in the above solution.

For the recursive case: there is a selection from $S[1, \dots, i]$ with a sum of c *if and only if* there is a selection from $S[1, \dots, i - 1]$ with a sum of either $c - S[i]$ (and $S[i]$ is selected) or of c (and $S[i]$ is not selected). In other words, $P(c, i)$ is true if and only if $P(c - S[i], i - 1)$ is true or $P(c, i - 1)$ is true (or both).

Note that the case of checking $s_i > c$ is handled in this recursive case because $P(c - s_i, i - 1)$ would return False according to the new base case as $c - s_i < 0$.

+1 Correct base case when $c = 0$

+0.5 Correct justification why it is True.

+1 Correct base case when $c > 0, i = 0$

+0.5 Correct justification why it is False.

+2 Either give base case for $c < 0$ (or) ignoring the item with $s_i > c$ leading to $P(c, i - 1)$ term.

+1 Correct recursive case first part ($P(c - s_i, i - 1)$)

+1 Correct recursive case second part ($P(c, i - 1)$)

+1 Use logical or to combine the results

+2 For correct justification of the recurrence.

- (3 pts) (b) Briefly state, with justification, the values of c and i to plug into $P(c, i)$, to determine if it is possible to split the candy into two piles having equal amounts of sugar.

Solution:

Let $\sigma = \text{sum}(S) = \sum_{i=1}^n S[i]$ be the sum of all the numbers in the array. If σ is even, then the final answer is $P(\sigma/2, n)$; otherwise the answer is false. This can be seen as follows: the numbers can be partitioned into two ‘piles’ having equal totals if and only if there is a selection from the *entire* array $S[1, \dots, n]$ with a sum of $\sigma/2$, corresponding to the numbers in one of the piles. (The leftover array elements also sum to $\sigma/2$, and correspond to the other pile.)

+0.5 Computes $\text{sum}(S)$

+1.5 Uses $c = \text{sum}(S)/2$

+1 Uses $i = n$

- (5 pts) (c) Give pseudocode for a dynamic-programming algorithm that determines whether the candy can be divided into two piles that have the same amount of sugar. Analyze the running

time.

Note that this is not an efficient algorithm (in fact, there is no known efficient algorithm for this problem!). However, if each s_i is upper bounded by a constant, then your algorithm should be efficient.

Solution:

The following algorithm implements the recurrence from the previous part, filling a $\lfloor \sigma/2 + 1 \rfloor$ -by- $(n + 1)$ table from the bottom up, then returns the table value given in the previous part.

```

1: function PARTITIONABLE( $S[1, \dots, n]$ )
2:   let  $\sigma = \text{sum}(S)$ 
3:    $t = \lfloor \sigma/2 \rfloor$ 
4:   initialize table $[0, \dots, t][0, \dots, n]$  // convention: table $[c][i] = \text{false}$  for  $c < 0$ 
5:   table $[0][0, \dots, n] = \text{true}$  // base cases
6:   for  $i = 1, \dots, n$  do
7:     for  $c = 1, \dots, t$  do
8:       table $[c, i] = \text{table}[c, i - 1] \vee \text{table}[c - S[i]][i - 1]$ 
9:   if  $\sigma$  is even then return table $[t, n]$ 
10:  else return false

```

The algorithm fills the table so that $\text{table}[c][i] = P(c, i)$. It does so one ‘row’ at a time, for increasing i , which is a valid order because the values for index i depend only on the values for index $i - 1$. Finally, it outputs the appropriate table value as described in the previous part.

By inspection of the loops (each iteration of the inner loop takes constant time), the overall running time is $\Theta(n \cdot t) = \Theta(n \cdot \text{sum}(S))$.

- +1 Correct implementation of the base case in (a) (regardless of whether (a) is correct)
- +1 Correct implementation of the recursive case in (a) (regardless of whether (a) is correct)
- +1 Return the correct final answer based on even or odd sum.
- +1 If the pseudocode fills in table in correct order
- +1 Correct runtime analysis

- (4 pts) (d) Not eating the candy is not an option! Given the completed DP table from the previous part, describe in clear English how you would find the *smallest possible difference* in sugar between two piles.

Solution:

To find the smallest possible difference for a partition into two piles, we simply search

for the maximum value of $c \leq t$ such that $\text{table}[c][n] = P(c, n)$ is true, which takes $O(t) = O(\text{sum}(S))$ time. This corresponds to a selection from $S[1, \dots, n]$ that sums to c , so the remaining elements sum to $\sigma - c$, for a difference of $\sigma - 2c$. (Maximizing c minimizes this difference.)

+1 Searches the table (outside of just the last element)

+1.5 Uses the maximum value of $c \leq t$ such that $\text{table}[c][n] = P(c, n)$ is true

+1.5 States that the smallest possible difference is $\sigma - 2c$

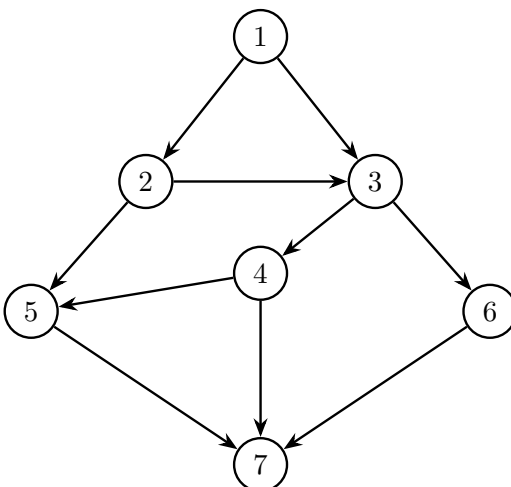
6. **Pebble games.** Many two-player strategic games, like the several variants of [Nim](#), can be modeled as follows. There is a directed acyclic graph $G = (V, E)$ presented in “topological” order: the vertices are labeled as $V = \{1, \dots, n\}$, and every edge $(u, v) \in E$ has $u < v$. Moreover, every vertex $i < n$ has at least one outgoing edge.

Two players, called Maize and Blue, take turns in the following game on the graph. They start with a pebble at vertex 1, and Maize plays first. On each turn, the active player (the player whose turn it is) chooses an edge $(u, v) \in E$ and moves the pebble from its current vertex u to a new vertex v . Unless $v = n$, the other player then becomes the active player with the pebble at v . If a player moves the pebble to vertex n (the final one), then that player wins the game.

It turns out that, for any fixed graph G , one player can be *guaranteed* a win by playing “perfectly”—regardless of what their opponent does. In this problem you will develop a dynamic-programming algorithm that, given the graph G as input, determines *which* player can be guaranteed a win, along with *how* that player can play perfectly.

(2 pts)

- (a) In the following graph, there are 7 nodes numbered in topological order. Suppose the pebble is at vertex 1. Determine whether or not the active player (the player who is about to move the pebble) can guarantee a win regardless of what the other player does in future turns. Justify your answer.



Solution: They can guarantee a win by moving the pebble to node 3. Then the opponent must move the pebble to either 4 or 6, at which point the first player wins by moving the pebble to 7.

+1 For mentioning that player 1 has a winning strategy.

+1 Explaining the strategy correctly. No partial credit here.

- (2 pts) (b) List all vertices in the above graph for which, if the pebble is at that vertex, the active player can guarantee a win. You do not need to justify your answer.

Solution: The vertices at which an active player have a winning strategy are 1,2,4,5,6.

+0.5 Give 0.5 for each of 2,4,5,6 ignoring node 1.

-0.5 Penalize 0.5 for each of 3,7.

- (8 pts) (c) For a given graph, define $W(i) = \text{true}$ if the active player can be guaranteed a win when the pebble is at vertex i . (Otherwise, $W(i) = \text{false}$.) Derive, with justification, a recurrence relation for $W(i)$. Remember to include appropriate base case(s).

Notation: For representing the Boolean operators $\vee, \wedge$ across many variables, you may use the notation $\bigvee, \bigwedge$. For example $\bigwedge_{i=1}^k x_i = x_1 \wedge x_2 \wedge \cdots \wedge x_k$.

Solution: The base case and recurrence for W is as follows (the $\bigvee_{(i,j) \in E}$ notation is the logical OR over all edges (i, j) from i to one of its out-neighbors j):

$$W(i) = \begin{cases} \text{false} & \text{if } i = n, \\ \bigvee_{(i,j) \in E} \neg W(j) & \text{otherwise.} \end{cases}$$

(In fact, a separate base case of $i = n$ is not even technically needed here, since the OR of zero terms is defined to be false. Also, note that the above “OR of NOTs” is also equivalent to a “NOT of ANDs,” by De Morgan’s laws.)

The justification for the base case is: if the pebble is at vertex n , then the active player has already lost (it cannot move), so there is no possible way for the active player to win, and therefore $W(n) = \text{false}$.

For the recursive case: if the pebble is at vertex $i < n$, then the active player can guarantee a win exactly when it can make a move to put its opponent in a “losing position,” i.e., one for which there is no winning strategy. In other words, i is a winning position exactly when there is some edge $(i, j) \in E$ for which the opposing player *cannot* be guaranteed a win from j , i.e., $W(j) = \text{false}$.

It is also worth noticing that, according to the recurrence, if $W(i) = \text{false}$ then $W(j) = \text{true}$ for *every* edge $(i, j) \in E$. In other words, a player at a “losing position” i (where $W(i) = \text{false}$) has either already lost (because $i = n$), or has no choice but to

move the pebble to a vertex j that is a “winning position” (i.e., $W(j) = \text{true}$) for its opponent.

+1.5 For correctly giving the base case with justification why it is false.

+1.5 For correctly identifying that active player has to force the other player to lose.

+2.5 For stating the correct recurrence, penalize 1 points for mistakes in writing the recurrence if the justification is correct.

+2.5 Giving correct justification for the recurrence.

- (5 pts) (d) The algorithm should take as input a directed graph $G = (V, E)$ and determine which of the two players has a guaranteed winning strategy. Design a dynamic programming algorithm that runs in $O(m)$ time, where $m = |E|$. Provide pseudocode for the algorithm and justify its running time.

The input graph $G = (V, E)$ is represented via *adjacency lists*, i.e., for each vertex $i \in V$ there is a list of all its *out-neighbors*: vertices j such that $(i, j) \in E$.

Solution: Using the recurrence we get the following dynamic programming algorithm. Notice that the algorithm fills the table W from “back” to “front,” which is valid because every edge $(i, j) \in E$ has $i < j$, so when setting $W[i]$ all the needed values of $W[j]$ have already been correctly set. Also, the final output value $W[1]$ is correct, because by definition, it is true if and only if the first player (Maize) can be guaranteed a win at the start of the game (when the pebble is at position 1). The running time is $O(m)$, where $m = |E|$ is the number of edges in the graph, because over *all* the iterations of the outer loop in total, the inner loop examines each edge in the graph exactly once.

```

1: function WINNING( $V[1, \dots, n], E$ )
2:   initialize table  $W[1, \dots, n]$ 
3:    $W[n] = \text{false}$ 
4:   for  $i = n - 1$  down to 1 do
5:      $W[i] \leftarrow \text{false}$ 
6:     for all edges  $(i, j) \in E$  (walk the adjacency list of  $i$ ) do
7:       if  $W[j] = \text{false}$  then  $W[i] \leftarrow \text{true}$ 
8:   return  $W[1]$  // whether Maize can be guaranteed a win from the start

```

+1 The pseudocode / justification should mention which player wins the game (either Maize or Blue) based on $W[1]$.

+0.5 Mentions base case correctly in the pseudocode

+1 For filling the table in the correct order. It should be from $n - 1$ to 1.

- +1 For truthful implementation of their recurrence. It's ok if they copy the expression for the recurrence into the pseudocode instead of explicitly iterating through all of the incident edges in the for loop.
- +1.5 For correct analysis of running time explaining how they get $O(m)$.

- (5 pts) (e) Describe, in clear and precise English, how to extend your algorithm from the previous part so that it also outputs a “winning strategy” for whichever player has one. This should be represented as an array specifying, for each vertex u , where the player should move the pebble if it is at vertex u on that player's turn.

Solution: The extended algorithm will allocate and fill an array $\text{Next}[1, \dots, n]$, where $\text{Next}(i)$ indicates where the active player can move to, from position i , in a winning strategy. (If there is no winning strategy from position i , then $\text{Next}[i]$ can be some failure symbol, or left undefined.)

For each vertex $i < n$ where $W(i) = \text{true}$, there must exist at least one edge (i, j) where $W(j) = \text{false}$, due to the recurrence for $W(i)$ in the previous parts.

So, when we set $W[i] = \text{true}$ in the algorithm from the previous part, we also set $\text{Next}[i] = j$ for the corresponding value of j . When we initially set $W[i] = \text{false}$, we leave $\text{Next}[i]$ undefined.

The correctness of the Next array can be seen as follows. If either player—say, Maize—is at a position i where $W[i] = \text{true}$, then letting $j = \text{Next}[i]$, we have that $W[j] = \text{false}$. So, moving to vertex j leaves the opponent Blue in a “losing position,” i.e., one in which any move Blue makes goes to a vertex k where $W[k] = \text{true}$. Then Maize is again at a winning position, and can repeat the above until the game ends with a win for Maize.

This modification preserves the $O(n + m)$ running time of the previous algorithm because it does just $O(1)$ additional work when setting each $W[i]$.

- +1 Mentioning that for a node i with $W(i) = \text{false}$, there is no winning move.
- +2 Correctly identifying that the player should move the pebble to the node which has false.
- +2 Correctly explaining how the algorithm is extended to populate the “winning strategy”.

7. **Spell Checker** Imagine that you are building a spellchecker for a word processor. When the spellchecker encounters an unknown word, you want it to suggest a word in its dictionary that the user might have meant (perhaps they made a typo). One way to generate this suggestion is to measure how “close” the typed word A is to a particular word B from the dictionary, and suggest the closest of all dictionary words. There are many ways to measure closeness; in this problem we will consider a measure known as the *edit distance*, denoted $\text{EDIT-DIST}(A, B)$.

In more detail, given strings A and B , consider transforming A into B , via character *insertions* (i), *deletions* (d), and *substitutions* (s). For example, if $A = \text{ALGORITHM}$ and $B = \text{ALTRUISTIC}$, then

one way of transforming A into B is via the following operations:

A	L	G	O	R		I		T	H	M
A	L	T		R	U	I	S	T	I	C
		s	d		i		i		s	s

EDIT-DIST(A, B) is the minimum cost of transforming string A to string B , given the following three positive numbers:

- c_i , the cost to insert a character;
- c_d , the cost to delete a character;
- c_s , the cost to substitute a character.

In this problem you will give and analyze an efficient dynamic programming algorithm that, on input of two strings A, B , computes EDIT-DIST(A, B) with the costs for insertion, deletion and substitution as c_i, c_d and c_s respectively.

- (8 pts) (a) Give a recurrence relation for edit distance EDIT-DIST(A, B) and justify its correctness. Remember to include base case(s).

Hint: the recurrence relation for LCS is a good place to look for inspiration.

Solution: Letting $m = |A|$ and $n = |B|$, we can represent the strings as character arrays: $A = A[1, \dots, m]$ and $B = B[1, \dots, n]$.

Let us define $\mathbf{ED}(i, j)$ be the edit distance of length i prefix of A and length j prefix of B concisely, EDIT-DIST($A[1, \dots, i], B[1, \dots, j]$).

The value we ultimately want to compute is $\mathbf{ED}(m, n)$, because it equals EDIT-DIST(A, B).

We give a recurrence relation for $\mathbf{ED}$. There are two **base cases**:

- If $i = 0$, then the length- i prefix of A is the empty string, so we must insert j characters into it in order to transform it into the length- j prefix of B . Hence, the edit distance is $\mathbf{ED}(0, j) = c_i \times j$.
- If $j = 0$, then the length- j prefix of B is the empty string, so we must delete all i characters from the length- i prefix of A in order to transform it into B . Hence, the edit distance is $\mathbf{ED}(i, 0) = c_d \times i$.

For obtaining the recurrence relation, there are (only) three possible ways on how one can (optimally) transform $A[1, \dots, i]$ to $B[1, \dots, j]$.

1. (optimally) transform $A[1, \dots, i]$ to $B[1, \dots, j-1]$, then *insert* $B[j]$ at the end;
2. (optimally) transform $A[1, \dots, i-1]$ to $B[1, \dots, j]$, then *delete* $A[i]$;
3. (optimally) transform $A[1, \dots, i-1]$ to $B[1, \dots, j-1]$ then,
 - either leave $A[i]$ unchanged, if it matches $B[j]$
 - (or) substitute $A[i]$ with $B[j]$ if they are different.

This yields the recurrence:

$$\mathbf{ED}(i, j) = \min \begin{cases} c_i + \mathbf{ED}(i, j - 1), \\ c_d + \mathbf{ED}(i - 1, j), \\ \mathbf{ED}(i - 1, j - 1) & \text{if } A[i] = B[j], \\ c_s + \mathbf{ED}(i - 1, j - 1) & \text{if } A[i] \neq B[j]. \end{cases}$$

Alternate solutions: It is also ok to omit “if $A[i] \neq B[j]$ ” because if $A[i] = B[j]$ then the substitution case will never be the min anyways:

$$\mathbf{ED}(i, j) = \min \begin{cases} c_i + \mathbf{ED}(i, j - 1), \\ c_d + \mathbf{ED}(i - 1, j), \\ \mathbf{ED}(i - 1, j - 1) & \text{if } A[i] = B[j], \\ c_s + \mathbf{ED}(i - 1, j - 1). \end{cases}$$

It is also possible to rigorously prove that leaving $A[i]$ unchanged is optimal for $\mathbf{ED}(i, j)$ when $A[i] = B[j]$. This allows the $A[i] = B[j]$ case to be simplified to a single case, but does not change the asymptotic runtime of the final algorithm. That is, another correct recurrence is:

$$\mathbf{ED}(i, j) = \begin{cases} \mathbf{ED}(i - 1, j - 1) & \text{if } A[i] = B[j], \\ \min \begin{cases} c_i + \mathbf{ED}(i, j - 1), \\ c_d + \mathbf{ED}(i - 1, j), \\ c_s + \mathbf{ED}(i - 1, j - 1) \end{cases} & \text{if } A[i] \neq B[j] \end{cases}$$

However, this recurrence requires a trickier proof of correctness.

-
- +1 Base case $\mathbf{ED}(0, i)$ with justification.
 - +1 Base case $\mathbf{ED}(i, 0)$ with justification.
 - +1 Correct recurrence for the insertion case, $c_i + \mathbf{ED}(i, j - 1)$
 - +0.5 Giving justification for the insertion case.
 - +1 Correct recurrence for the deletion case, $c_d + \mathbf{ED}(i - 1, j)$
 - +0.5 Giving justification for the deletion case.
 - +1 Correct recurrence for the same letter case, $\mathbf{ED}(i - 1, j - 1)$
 - +0.5 Giving justification for the same letter case.
 - +1 Correct recurrence for the substitution case, $c_s + \mathbf{ED}(i - 1, j - 1)$
 - +0.5 Giving justification for the substitution case.

- (4 pts) (b) Consider the translation of your recurrence into a dynamic programming algorithm. Analyze the running time. While justifying your answer, be sure to include a description of the order in which the DP table should be filled, and where the final answer is located. You are not required to give explicit pseudocode (though you may if you wish).

Solution: To compute an answer in a bottom-up fashion using this recurrence, we construct a $(m + 1) \times (n + 1)$ table $M[0, \dots, m][0, \dots, n]$, where $M[i][j]$ will hold the value of $\mathbf{ED}(i, j)$. One possible way to fill the table follows:

1. Fill in $M[i][0]$ and $M[0][j]$ as specified by the base cases.
2. For $i = 1, \dots, m$
 - (a) For $j = 1, \dots, n$, fill in $M[i][j]$ as specified by the above recurrence.

Observe that filling in the table in this order ensures that for each cell $M[i][j]$, its “dependencies” are calculated and filled in before it. That is, $M[i][j-1]$, $M[i-1][j]$, $M[i-1][j-1]$ are set before $M[i][j]$ is to be set.

(Note that in line (b), it does not matter that the outer loop corresponds to i and the inner loop corresponds to j . We could swap the inner and outer loops, and the algorithm would still be correct.)

There are $O(nm)$ entries to fill in the table, and each entry can be computed in $O(1)$ time, so the entire table (and $M[m][n]$ in particular) can be computed in $O(nm)$ time. We provide a bottom-up pseudocode implementation for reference (though this is not required for full credit).

Input: words A and B for which EDIT-DISTANCE needs to be computed
Output: EDIT-DISTANCE(A, B)

```

1: function EDIT-DISTANCE( $A, B$ )
2:   Initialize an  $(m + 1) \times (n + 1)$  table  $M$ 
3:   for  $i = 0, \dots, m$  do // Base cases
4:      $M[i][0] = c_d \times i$ 
5:   for  $j = 0, \dots, n$  do
6:      $M[0][j] = c_i \times j$ 
7:   for  $i = 1, \dots, m$  do // As noted above, lines 7 and 8 can be interchanged
8:     for  $j = 1, \dots, n$  do
9:       if  $A[i] = B[j]$  then
10:         $M[i][j] \leftarrow \min\{M[i-1][j-1], M[i][j-1] + c_i, M[i-1][j] + c_d\}$ 
11:       else
12:         $M[i][j] \leftarrow \min\{M[i-1][j-1] + c_s, M[i][j-1] + c_i, M[i-1][j] + c_d\}$ 
13:   return  $M[m][n]$ 

```

The final answer is located in the entry $M[m][n]$.

+0.5 Correct filling of base case $ED(0, i)$

+0.5 Correct filling of base case $ED(i, 0)$.

+1 Truthful implementation of the recurrence.

+1 Correct ordering for the recurrence given in previous part, even if the recurrence is wrong, DP table has to be filled correctly for that wrong recurrence.

+1 Mentioning $M[m][n]$ is the place where final answer is located.

- (3 EC pts) 8. **Optional extra credit: Wheel deliver.** Jett and Anthony have started a new unicycle-based meal delivery business called “Wheel Deliver” that works as follows. Residents of Ann Arbor submit their requests for a meal each day before noon using the Wheel Deliver App. In the afternoon, Jett and Anthony cook up a delicious meal in their kitchen and deliver the food on their unicycles.

Let n be the number of customers on a given day and $P = (p_1, \dots, p_n)$ be the list of locations of these customers as points in the 2D plane, sorted in the order in which the customers submitted their requests: p_1 is the location of the first person to place an order, and p_n is the location of the last person to place an order. Let p_0 denote the location of the kitchen from which Jett and Anthony depart. Let $d(p_i, p_j)$ denote the distance between p_i and p_j .

Jett and Anthony wish to split up the list of locations P , not necessarily evenly, so that each delivery is made by exactly one of them. In addition, “Wheel Deliver” has a fairness rule that goes like this: If customers at p_i and p_j for $i < j$ are assigned to the same unicycle rider, then the delivery to the customer at p_i must occur before the delivery to the customer at p_j —even if that makes the trip longer. (If p_i and p_j are assigned to different riders, then we don’t care

which delivery occurs first.) Subject to these constraints, the objective is to minimize the total distance travelled by the two unicycle riders.

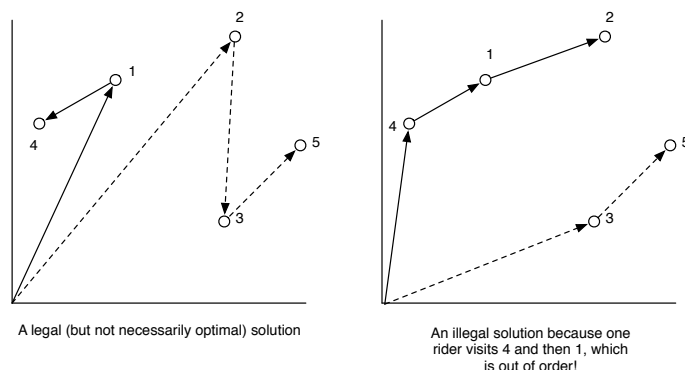


Figure 1: Two examples of routes for the two riders. The origin denotes the starting point p_0 . One rider's route is shown with solid lines and the other with dashed lines. The route on the left is fair because points are visited by increasing indices. The route on the right is not fair because, for example, point p_4 is visited before point p_1 .

Jett and Anthony have hired you to design an efficient algorithm that takes as input an array of points $P = (p_1, p_2, \dots, p_n)$, sorted by the times in which their orders arrived, and outputs the sequences of delivery points for the two riders.

Derive a recurrence relation, including base case(s), that is suitable for an efficient dynamic-programming solution for this problem. Briefly justify its correctness. You do not need to include pseudocode.

Warning: The most natural-seeming recurrences may have subtle errors!

Solution: For $i < j$, define $\text{Len}(i, j)$ to be the optimal total length of the two delivery routes assuming that the first j points receive delivery, and the two riders respectively end up at points i and j at the end. The base case is $i = 0, j = 1$; here the optimum solution is simply $\text{Len}(0, 1) = d(p_0, p_1)$. (One could also have a base case of $i = j = 0$ with $\text{Len}(0, 0) = 0$, though this deviates from the convention that $i < j$.) For the recursive case, we claim that

$$\text{Len}(i, j) = \begin{cases} \text{Len}(i, j-1) + d(p_{j-1}, p_j) & \text{if } i < j-1 \\ \min_{0 \leq k < i} \{ \text{Len}(k, i) + d(p_k, p_j) \} & \text{otherwise.} \end{cases}$$

The justification for correctness is as follows. For the recursive case, we consider two cases: In the first case, $i < j-1$. In that case, we know that the rider that ended up at p_j must have come from p_{j-1} so $\text{Len}(i, j) = \text{Len}(i, j-1) + d(p_{j-1}, p_j)$. The remaining case is that $i = j-1$. In that case, the rider that ended up at p_j must have come from some p_k where $k < i$. We don't know where that p_k is, so we'll try every possible option. Notice, that for any given $k < i$, the best solution in this case can be expressed as $\text{Len}(k, i) + d(p_k, p_j)$. Therefore, in this case our solution is $\min_{0 \leq k < i} \text{Len}(k, i) + d(p_k, p_j)$. Finally, the optimal

total distance is $\min_{0 \leq k < i} \text{Len}(i, n)$. By storing the minimizing choice in each recursive case, the two delivery routes can be recovered by backtracking.

There may be other valid recurrences, but the following two are NOT valid recurrences:

$$\text{Len}(i, j) = \min \begin{cases} \text{Len}(i-1, j) + d(p_{i-1}, p_i) \\ \text{Len}(i, j-1) + d(p_{j-1}, p_j) \end{cases}.$$

$$\text{Len}(i, j) = \begin{cases} \text{Len}(i, j-1) + d(p_{j-1}, p_j) & \text{if } i < j-1 \\ \min\{\text{Len}(i-1, j) + d(p_{i-1}, p_i), \text{Len}(i-1, i) + d(p_{i-1}, p_j)\} & \text{if } i = j-1. \end{cases}$$

+1.5 For correct recurrence. Be strict here.

+1.5 For correct justification. Be lenient here if the recurrence is correct.